Buổi 9

Tính kế thừa (Inheritance)

Học phần: Lập trình hướng đối tượng

Nội dung bài học

1. Lớp cơ sở và lớp dẫn xuất

1.1 Khái niệm kế thừa
1.2 Thành viên protected
1.3 Ví dụ phân cấp Point → Circle → Cylinder

2. Các vấn đề của đơn kế thừa

2.1 Định nghĩa lại thành viên lớp cơ sở
2.2 Ví dụ định nghĩa lại: Employee và HourlyWorker
2.3 Kế thừa public, protected và private
2.4 Constructor và destructor của lớp dẫn xuất
2.5 Chuyển đổi giữa đối tượng, con trỏ lớp cơ sở và lớp dẫn xuất
2.6 Kế thừa hay chứa

3. Đa kế thừa và lớp cơ sở ảo

3.1 Đa kế thừa
3.2 Lớp cơ sở ảo
3.3 Giải pháp: kế thừa ảo

4. Tổng kết và thực hành

4.1 Tóm tắt buổi 9
4.2 Bài thực hành buổi 9 – Phân cấp Point → Circle → Cylinder
4.3 Câu hỏi và bài tập

Kế thừa

Kế thừa là cơ chế tạo **lớp mới** (lớp **dẫn xuất** – derived class, lớp con) từ **lớp đã có** (lớp **cơ sở** – base class, lớp cha). Lớp dẫn xuất **thừa hưởng** các thành viên dữ liệu và hàm thành viên của lớp cơ sở, đồng thời **bổ sung** thành viên mới hoặc **định nghĩa lại** hàm của lớp cơ sở. Quan hệ: lớp dẫn xuất **"là một"** (is-a) lớp cơ sở.

Lớp cơ sở	Các lớp dẫn xuất
HinhHoc	HinhTron, HinhChuNhat, TamGiac
NhanVien	NVBienChe, NVHopDong, NVThoiVu
TaiKhoan	TKTietKiem, TKThanhToan
PhuongTien	XeMay, OTo, MayBay
Point	Circle (→ Cylinder)

- Mỗi đối tượng lớp dẫn xuất **cũng là** một đối tượng lớp cơ sở (hình tròn là một hình học), nhưng **không** ngược lại.

- **Cú pháp:**

```
class LopDanXuat : kieu_ke_thua LopCoSo
{  // thành viên bổ sung
};
// kieu_ke_thua: public | protected | private
class Circle : public Point { ... };
```

- **Lợi ích:**
  - **Tái sử dụng** mã đã kiểm thử của lớp cơ sở.
  - **Mở rộng** chức năng mà **không sửa** lớp cơ sở.
  - Tổ chức lớp thành **cây phân cấp** phản ánh thực tế.
  - Là nền tảng của **đa hình** (buổi 10).
- **Kế thừa trực tiếp – gián tiếp:** Cylinder kế thừa **trực tiếp** Circle, **gián tiếp** Point.
- **Đơn kế thừa** (một lớp cơ sở trực tiếp) và **đa kế thừa** (nhiều lớp cơ sở – mục 4).

1.2 Thành viên protected

- Lớp dẫn xuất **không** truy cập được thành viên **private** của lớp cơ sở (dù chúng vẫn nằm trong đối tượng dẫn xuất).
- **protected:** mức truy cập **trung gian** – như private đối với bên ngoài, nhưng **lớp dẫn xuất** truy cập được.

Thành viên của lớp cơ sở	Hàm của chính lớp	Hàm của lớp dẫn xuất	Bên ngoài (main…)
public	Có	Có	Có
protected	Có	**Có**	Không
private	Có	**Không**	Không

```
class Point
{  public:
      Point(double a = 0, double b = 0);
      double GetX() const { return X; }
      double GetY() const { return Y; }
   protected:
      double X, Y;       // lớp dẫn xuất dùng trực tiếp
};
```

Ví dụ 9.1 – Thông báo lỗi của g++

```
class Base { public: int pub; protected: int pro;
             private: int pri; };
class D1 : public Base
{  public: void f() { pub = 1; pro = 2; pri = 3; }  };
int main() { D1 a;  a.pro = 1; }
```

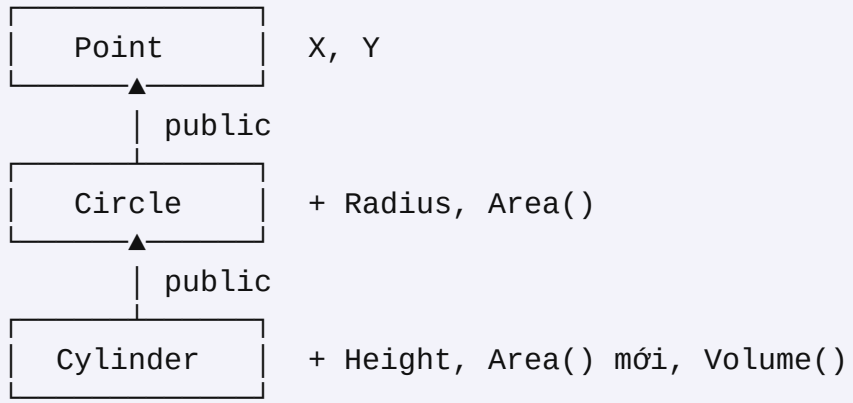
```
error: 'int Base::pri' is private within this context
error: 'int Base::pro' is protected within this context
```

Cân nhắc khi dùng protected

protected làm **yếu** tính đóng gói: thay đổi thành viên protected có thể làm hỏng **mọi** lớp dẫn xuất. Cách an toàn hơn: để dữ liệu **private** và cung cấp hàm get/set **protected** cho lớp dẫn xuất.

1.3 Ví dụ phân cấp Point → Circle → Cylinder (giáo trình)

```
class Point
{  public:
      Point(double a = 0, double b = 0) { X = a; Y = b; }
      double GetX() const { return X; }
   protected:
      double X, Y;
};
class Circle : public Point          // Circle là một Point
{  public:
      Circle(double r = 0, double a = 0, double b = 0)
         : Point(a, b)               // gọi constructor cơ sở
      {  Radius = (r >= 0) ? r : 0;  }
      double Area() const { return 3.14159*Radius*Radius; }
   protected:
      double Radius;
};
class Cylinder : public Circle       // Cylinder là một
Circle
{  public:
      Cylinder(double h = 0, double r = 0,
               double a = 0, double b = 0)
         : Circle(r, a, b)
      {  Height = (h >= 0) ? h : 0;  }
      double Area() const;           // định nghĩa lại
      double Volume() const { return Circle::Area() * Height;
}
   private:
      double Height;
};
```



- Đối tượng Cylinder chứa **X, Y** (từ Point), **Radius** (từ Circle), **Height** (của riêng nó).
- Cylinder gọi được `GetX()`, `GetRadius()` – kế thừa từ các lớp cha.
- Circle dùng trực tiếp `X`, `Y` (protected); Cylinder dùng trực tiếp `Radius`.

2.1 Định nghĩa lại thành viên lớp cơ sở

- Lớp dẫn xuất có thể định nghĩa hàm **cùng tên, cùng tham số** với lớp cơ sở ⇒ hàm của lớp dẫn xuất **che** (override/redefine) hàm của lớp cơ sở. Muốn gọi bản của lớp cơ sở: dùng `LopCoSo::Ham()`.

```
// Diện tích toàn phần hình trụ = 2 đáy + xung quanh
double Cylinder::Area() const
{  return 2 * Circle::Area()           // gọi bản của Circle
        + 2 * 3.14159 * Radius * Height;
}
int main()
{  Cylinder cyl(10, 3.3, 2, 3);        // h = 10, r = 3.3
   cout << "X = " << cyl.GetX()
        << ", Radius = " << cyl.GetRadius() << endl;
   cout << "Area = " << cyl.Area()
        << ", Circle area = " << cyl.Circle::Area()
        << ", Volume = " << cyl.Volume() << endl;
}
```

```
X = 2.00, Radius = 3.30
Area = 275.77, Circle area = 34.21, Volume = 342.12
```

- `cyl.Area()` gọi **Cylinder::Area** (diện tích toàn phần 275.77).
- `cyl.Circle::Area()` – gọi tường minh bản của lớp cơ sở (diện tích đáy 34.21).
- Trong `Cylinder::Area`, nếu viết `Area()` thay vì `Circle::Area()` ⇒ gọi **chính nó ⇒ đệ quy vô hạn**.

Che giấu tên

Nếu lớp dẫn xuất định nghĩa hàm cùng tên nhưng **khác tham số**, mọi bản của lớp cơ sở cùng tên đều bị **che** (không phải đa năng hóa) – phải gọi qua `LopCoSo::` hoặc khai báo `using LopCoSo::Ham;`.

2.2 Ví dụ định nghĩa lại: Employee và HourlyWorker (giáo trình)

```
class Employee
{  public:
      Employee(const char *f, const char *l)
      {  FirstName = new char[strlen(f) + 1];
         strcpy(FirstName, f);
         LastName = new char[strlen(l) + 1];
         strcpy(LastName, l);
      }
      ~Employee() { delete[] FirstName; delete[] LastName; }
      void Print() const
      {  cout << FirstName << ' ' << LastName;  }
   private:
      char *FirstName, *LastName;
};
class HourlyWorker : public Employee
{  public:
      HourlyWorker(const char *f, const char *l,
                   double h, double w)
         : Employee(f, l)
      {  Hours = h;  Wage = w;  }
      double GetPay() const
      {  if (Hours <= 40) return Wage * Hours;
         return 40 * Wage + (Hours - 40) * Wage * 1.5;  }
      void Print() const;           // định nghĩa lại
   private:
      double Hours, Wage;
};
```

```
void HourlyWorker::Print() const
{  cout << "HourlyWorker::Print() is executing\n\n";
   Employee::Print();       // dùng lại bản lớp cơ sở
   cout << " is an hourly worker with pay of $"
        << GetPay() << endl;
}
int main()
{  HourlyWorker h("Bob", "Smith", 40.0, 10.00);
   h.Print();
   HourlyWorker k("Ann", "Lee", 45.0, 10.00);
   k.Print();
}
```

```
HourlyWorker::Print() is executing

Bob Smith is an hourly worker with pay of $400
HourlyWorker::Print() is executing

Ann Lee is an hourly worker with pay of $475
```

- **Phân tích:**
  - FirstName, LastName là **private** của Employee ⇒ HourlyWorker **không** in trực tiếp được; phải gọi `Employee::Print()` – vẫn giữ đóng gói.
  - Ann làm 45 giờ: 40 × 10 + 5 × 10 × 1.5 = **475**.
  - Destructor của Employee **tự động** giải phóng họ tên khi HourlyWorker bị hủy.

2.3 Kế thừa public, protected và private

- Kiểu kế thừa quyết định mức truy cập **trong lớp dẫn xuất** của các thành viên thừa hưởng:

Thành viên ở lớp cơ sở	Kế thừa public	Kế thừa protected	Kế thừa private
public	→ **public**	→ protected	→ private
protected	→ protected	→ protected	→ private
private	Không truy cập được	Không truy cập được	Không truy cập được

- **public** (dùng phổ biến nhất): giữ nguyên giao diện – thể hiện đúng quan hệ "**là một**"; con trỏ cơ sở trỏ được tới đối tượng dẫn xuất.
- **protected / private:** thể hiện "**được cài đặt dựa trên**" – lớp dẫn xuất dùng lại cài đặt nhưng **không** công bố giao diện của lớp cơ sở ra ngoài.
- Mặc định: `class D : Base` là **private**; `struct D : Base` là public.

Ví dụ 9.2 – Thử với g++

```
class D2 : protected Base { };
D2 b;
b.pub = 1;
// error: 'int Base::pub' is inaccessible within this
context
```

- pub là public trong Base, nhưng qua kế thừa protected trở thành **protected** trong D2 ⇒ main không truy cập được.

Ví dụ 9.3 – Khi nào dùng private

`class NganXep : private DanhSach` – ngăn xếp được **cài đặt bằng** danh sách nhưng **không** cho bên ngoài gọi các hàm "chèn giữa", "xóa giữa" của danh sách; NganXep tự công bố Push/Pop. (Thường thay bằng **chứa** một đối tượng DanhSach – composition.)

2.4 Constructor và destructor của lớp dẫn xuất

- Constructor **không** được kế thừa. Constructor lớp dẫn xuất phải **gọi constructor lớp cơ sở** trong **danh sách khởi tạo**; nếu không ghi, constructor **mặc định** của lớp cơ sở được gọi.
- **Thứ tự gọi:**
  - **Tạo:** constructor lớp **cơ sở** → (đối tượng thành viên) → thân constructor lớp **dẫn xuất**. Đi từ **gốc** của cây kế thừa xuống.
  - **Hủy: ngược lại**: destructor lớp dẫn xuất → … → lớp cơ sở.

Ví dụ 9.4 – In thông báo trong mọi constructor/destructor

```
Cylinder cyl(10, 3.3, 2, 3);
...
cout << "Ket thuc main\n";
```

```
  Point constructor
  Circle constructor
  Cylinder constructor
  ...
Ket thuc main
  Cylinder destructor
  Circle destructor
  Point destructor
```

- **Giải thích:** phần Point của đối tượng phải được khởi tạo **trước** để Circle dùng được X, Y; phần Cylinder hủy **trước** vì có thể còn dùng dữ liệu của lớp cơ sở.
- **Truyền tham số lên lớp cơ sở:**

```
Cylinder(double h, double r, double a, double b)
   : Circle(r, a, b)     // Circle lại gọi Point(a, b)
{  Height = h;  }
```

- Lớp cơ sở **không có** constructor mặc định mà constructor dẫn xuất không gọi tường minh ⇒ lỗi: **no matching function for call to 'Point::Point()'**.
- Destructor lớp dẫn xuất **chỉ** dọn dẹp phần của nó; destructor lớp cơ sở **tự động** được gọi sau đó (không gọi tường minh).

2.5 Chuyển đổi giữa đối tượng, con trỏ lớp cơ sở và lớp dẫn xuất

- **Lớp dẫn xuất ⇒ lớp cơ sở (hướng lên – upcasting):** luôn hợp lệ và **ngầm định** (với kế thừa public) – vì Cylinder **là một** Point.

```
Cylinder cyl(10, 3.3, 2, 3);
Point *pp = &cyl;            // con trỏ cơ sở ← đối tượng dẫn
xuất
cout << "*pp = " << *pp;     // [2.00, 3.00]
Point &rp = cyl;             // tham chiếu cũng được
Circle c = cyl;              // ĐỐI TƯỢNG: bị "cắt"
cout << c.Area();            // 34.21
```

- Qua `pp` chỉ truy cập được **phần Point** (X, Y, GetX…); `pp->Volume()` ⇒ lỗi biên dịch.
- **Cắt đối tượng (slicing):** gán đối tượng dẫn xuất cho đối tượng cơ sở ⇒ chỉ sao chép phần cơ sở, phần riêng (Height) bị **mất**.

- **Lớp cơ sở ⇒ lớp dẫn xuất (hướng xuống – downcasting): không** ngầm định; phải ép kiểu **tường minh** và người lập trình tự chịu trách nhiệm:

```
Circle *cp = static_cast<Circle *>(pp);
cout << "*cp: " << *cp;
// Center = [2.00, 3.00]; Radius = 3.30
```

- Đúng vì pp **thực sự** trỏ tới một Cylinder (cũng là Circle). Nếu pp trỏ tới một **Point thuần** mà ép sang Circle ⇒ truy cập `Radius` **không tồn tại** ⇒ giá trị rác.
- Buổi 10: `dynamic_cast` kiểm tra kiểu **lúc chạy**, trả về NULL nếu ép sai.

Chuyển đổi	Ngầm định?	An toàn?
Dẫn xuất* → Cơ sở*	Có	Có
Cơ sở* → Dẫn xuất*	Không (ép tường minh)	Chỉ khi thực sự trỏ tới đối tượng dẫn xuất
Đối tượng dẫn xuất → Đối tượng cơ sở	Có	Mất phần riêng (slicing)

2.6 Kế thừa hay chứa (is-a hay has-a)?

Quan hệ	Câu hỏi kiểm tra	Cài đặt	Ví dụ
Là một (is-a)	"B **là một** A?"	Kế thừa public	SinhVien là một Nguoi
Có một (has-a)	"B **có một** A?"	Thành viên là đối tượng	NhanVien có NgaySinh (Date)
Được cài đặt bằng	"B **dùng** A để làm việc?"	Chứa (ưu tiên) hoặc kế thừa private	NganXep dùng Mang

Ví dụ 9.5 – Thiết kế sai thường gặp

- `class Circle : public Point` – hình tròn "là một" điểm? Giáo trình dùng để minh họa kỹ thuật; về thiết kế, hợp lý hơn là Circle **có một** tâm (Point).
- `class XeHoi : public DongCo` – xe **không phải** là động cơ ⇒ dùng **chứa**.
- `class HinhVuong : public HinhChuNhat` – về toán học là đúng, nhưng nếu HinhChuNhat có `SetDai()`, `SetRong()` độc lập thì hình vuông bị phá vỡ ⇒ cần cân nhắc.

- **Nguyên tắc:** "Ưu tiên **chứa** hơn **kế thừa**" khi không chắc chắn; chỉ kế thừa public khi đối tượng dẫn xuất **dùng được ở mọi nơi** cần đối tượng cơ sở (nguyên lý thay thế Liskov).

```
// Thiết kế bằng chứa
class Circle2
{  public:
      Circle2(double r, double a, double b)
         : Center(a, b), Radius(r) {}
      double GetX() const { return Center.GetX(); }
   private:
      Point Center;        // có một tâm
      double Radius;
};
```

- So với kế thừa: Circle2 phải tự viết GetX() "chuyển tiếp" sang Center, nhưng **không** bị lộ các hàm không phù hợp của Point.

3.1 Đa kế thừa (Multiple Inheritance)

- Lớp dẫn xuất có **nhiều** lớp cơ sở trực tiếp: `class C : public A, public B { … };`

```
class NhanVien
{  public:
      NhanVien(int ma) : Ma(ma)
      {  cout << "  NhanVien " << Ma << endl;  }
      void In() const { cout << "NV " << Ma; }
   protected:
      int Ma;
};
class SinhVien
{  public:
      SinhVien(int ma) : Ma(ma)
      {  cout << "  SinhVien " << Ma << endl;  }
      void In() const { cout << "SV " << Ma; }
   protected:
      int Ma;
};
class TroGiang : public NhanVien, public SinhVien
{  public:
      TroGiang(int mnv, int msv)
         : SinhVien(msv), NhanVien(mnv)
      {  cout << "  TroGiang\n";  }
      void In() const
      {  NhanVien::In();  cout << " / ";
         SinhVien::In();  cout << endl;  }
};
```

```
TroGiang tg(101, 2024);
tg.In();
```

```
  NhanVien 101
  SinhVien 2024
  TroGiang
NV 101 / SV 2024
```

- Constructor các lớp cơ sở được gọi theo **thứ tự khai báo kế thừa** (NhanVien rồi SinhVien), **không** theo thứ tự trong danh sách khởi tạo.
- **Nhập nhằng tên:** cả hai lớp cơ sở đều có `Ma` và `In()` ⇒ trong TroGiang phải ghi rõ `NhanVien::Ma`, `SinhVien::In()`; gọi `tg.Ma` từ ngoài ⇒ lỗi **ambiguous**.
- **Ưu điểm:** mô hình hóa đối tượng thuộc nhiều loại. **Nhược điểm:** phức tạp, dễ nhập nhằng; Java/C# **không** cho đa kế thừa lớp (chỉ cho đa kế thừa giao diện).

3.2 Lớp cơ sở ảo (Virtual Base Classes)

- **Vấn đề "kim cương":** GV và SV cùng kế thừa Nguoi; TG kế thừa cả GV và SV ⇒ đối tượng TG chứa **hai bản** Nguoi.

```
class Nguoi
{  public:
      Nguoi(const char *t = "?")
      {  cout << "  Nguoi(" << t << ")\n";  Ten = t;  }
      const char *Ten;
};
class GV : public Nguoi
{  public: GV(const char *t) : Nguoi(t) { cout << "  GV\n"; }
};
class SV : public Nguoi
{  public: SV(const char *t) : Nguoi(t) { cout << "  SV\n"; }
};
class TG : public GV, public SV
{  public: TG(const char *t) : GV(t), SV(t)
           { cout << "  TG\n"; }  };
TG a("An");
```

```
  Nguoi(An)
  GV
  Nguoi(An)        ← bản Nguoi thứ hai!
  SV
  TG
```

- `a.Ten = "x";` ⇒ error: **request for member 'Ten' is ambiguous** (Ten nào – của GV hay của SV?).

```
          Nguoi    Nguoi
            |        |
           GV       SV
             \      /
               TG
```

- **Hậu quả của hai bản Nguoi:**
  - Tốn bộ nhớ, dữ liệu **trùng lặp** (Tên lưu hai lần, có thể **khác nhau** nếu sửa qua một nhánh).
  - Constructor Nguoi chạy **hai lần**.
  - Truy cập thành viên của Nguoi bị **nhập nhằng**; phải viết `a.GV::Ten` hoặc `a.SV::Ten`.
  - Chuyển `TG*` sang `Nguoi*` cũng nhập nhằng (Nguoi nào?).

3.3 Giải pháp: kế thừa ảo

- **Giải pháp – kế thừa ảo:** `class GV : virtual public Nguoi` ⇒ mọi lớp dẫn xuất qua nhiều đường **dùng chung một** bản Nguoi.

```
class GVv : virtual public Nguoi { … };
class SVv : virtual public Nguoi { … };
class TGv : public GVv, public SVv
{  public:
      TGv(const char *t)
         : Nguoi(t), GVv(t), SVv(t)   // gọi Nguoi TRỰC TIẾP
      {  cout << "  TGv\n";  }
};
TGv b("Binh");
cout << b.Ten;
```

```
  Nguoi(Binh)      ← chỉ MỘT lần
  GVv
  SVv
  TGv
Binh
```

```
          Nguoi
         /     \
      GVv      SVv
         \     /
           TGv
```

- Với lớp cơ sở ảo, lớp **cuối cùng** (TGv) chịu trách nhiệm gọi constructor của lớp cơ sở ảo; lời gọi `Nguoi(t)` trong GVv, SVv bị **bỏ qua**.

4.1 Tóm tắt buổi 9

Khái niệm	Ghi nhớ
Kế thừa	`class D : public B` – D "là một" B; tái sử dụng, mở rộng
protected	Lớp dẫn xuất truy cập được, bên ngoài thì không
Định nghĩa lại	Hàm cùng tên che hàm lớp cơ sở; gọi bản cơ sở bằng `B::Ham()`
Kiểu kế thừa	public giữ nguyên; protected/private thu hẹp quyền truy cập
Constructor/destructor	Không kế thừa; gọi B(…) trong danh sách khởi tạo; tạo cơ sở → dẫn xuất, hủy ngược lại
Chuyển đổi	D* → B* ngầm định; B* → D* phải ép, chỉ đúng khi thật sự là D; slicing khi gán đối tượng
Đa kế thừa	`class C : public A, public B`; coi chừng nhập nhằng tên
Lớp cơ sở ảo	`virtual public` – một bản chung trong mô hình kim cương

Câu hỏi nhanh

- `class C : public B`, B có `int x` private. Đối tượng C có chứa x không? → **Có** (chiếm bộ nhớ), nhưng hàm của C **không** truy cập trực tiếp được.
- Đối tượng Cylinder có sizeof bằng bao nhiêu (double 8 byte)? → **32** (X, Y, Radius, Height).

4.2 Bài thực hành buổi 9 – Phân cấp Point → Circle → Cylinder

Yêu cầu

- 1) Cài đặt Point, Circle, Cylinder như mục 1.3: constructor dùng danh sách khởi tạo, hàm get/set có kiểm tra, `<<` cho từng lớp.
- 2) Thêm lệnh in vào mọi constructor/destructor; tạo một Cylinder trong khối `{ }` và giải thích thứ tự in ra.
- 3) Định nghĩa lại `Area()` ở Cylinder (diện tích toàn phần), thêm `Volume()`; in diện tích đáy bằng `Circle::Area()`.
- 4) Thí nghiệm chuyển đổi: con trỏ `Point *` trỏ tới Cylinder; ép xuống `Circle *`; gán đối tượng Cylinder cho Circle (slicing) – in kết quả và giải thích.
- 5) Viết lại Circle theo cách **chứa** Point (Circle2) và so sánh hai thiết kế.
- 6) (Mở rộng) Phân cấp NhanVien → NVBienChe (hệ số lương), NVHopDong (lương theo ngày công); hàm `TinhLuong()` định nghĩa lại ở mỗi lớp.

Gợi ý kiểm tra

Với Cylinder(h = 10, r = 3.3, x = 2, y = 3): diện tích đáy **34.21**, diện tích toàn phần **275.77**, thể tích **342.12** (π = 3.14159, in 2 chữ số thập phân).

4.3 Câu hỏi và bài tập

Câu hỏi

- 1) Phân biệt private và protected. Vì sao nói protected làm yếu đóng gói?
- 2) Trình bày bảng quyền truy cập theo ba kiểu kế thừa public, protected, private.
- 3) Thứ tự gọi constructor/destructor khi tạo/hủy đối tượng lớp dẫn xuất (có đối tượng thành viên)?
- 4) Ép kiểu con trỏ lớp cơ sở sang lớp dẫn xuất khi nào an toàn? Slicing là gì?
- 5) Vấn đề kim cương trong đa kế thừa là gì? Lớp cơ sở ảo giải quyết thế nào?

Bài tập dự đoán kết quả (có đáp số)

```
class A { public: A() { cout << "A "; } ~A() { cout << "~A "; } };
class B { public: B() { cout << "B "; } ~B() { cout << "~B "; } };
class C : public A
{  public: C() { cout << "C "; } ~C() { cout << "~C "; }
   private: B b;
};
int main() { C c; cout << "| "; }
```

→ **Đáp số:** `A B C | ~C ~B ~A` – lớp cơ sở A trước, rồi thành viên b, rồi thân C; hủy ngược lại.